FACE Prep

Operating System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Q1

Save

Completed

Non-Preemptive Priority Scheduling algorithm

Write a C program to implement the Non-Preemptive Priority Scheduling algorithm. The program should accept number of processes their Arrival Time (AT), Burst Time (BT), and Priority for each process. The process with the highest priority (lowest numerical value) must be executed first among those that have arrived. Calculate the Average Waiting Time and Average Turnaround Time.

Input Format

The first line contains an integer representing the number of processes.

The next lines contain three integers for each process, where

- The first integer denotes the Arrival Time (AT)
- The second integer denotes the Burst Time (BT)
- The third integer denotes the Priority of the process.

A lower numerical value indicates a higher priority.

Output Format

Select Language : C (GCC 9.2.0)

```c
#include <stdio.h>

int main() {
    int n, i, time = 0, completed = 0;

    printf("Enter number of processes: \n");
    scanf("%d", &n);

    int at[n], bt[n], pr[n], ct[n], tat[n], wt[n], visited[n];

    for(i = 0; i < n; i++) {
        printf("Enter AT, BT and Priority for P%d: \n", i + 1);
        scanf("%d %d %d", &at[i], &bt[i], &pr[i]);
        visited[i] = 0;
    }

    while(completed < n) {
        int highest = 9999;
        int index = -1;

        for(i = 0; i < n; i++) {
            if(at[i] <= time && visited[i] == 0) {
                if(pr[i] < highest) {
                    highest = pr[i];
```

Test Cases 2

Run Sample Cases

Run All Cases

FACE Prep

Operating System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Q1

Save

Completed

Non-Preemptive Priority Scheduling algorithm

Write a C program to implement the Non-Preemptive Priority Scheduling algorithm. The program should accept number of processes their Arrival Time (AT), Burst Time (BT), and Priority for each process. The process with the highest priority (lowest numerical value) must be executed first among those that have arrived. Calculate the Average Waiting Time and Average Turnaround Time.

Input Format

The first line contains an integer representing the number of processes.

The next lines contain three integers for each process, where

- The first integer denotes the Arrival Time (AT)
- The second integer denotes the Burst Time (BT)
- The third integer denotes the Priority of the process.

A lower numerical value indicates a higher priority.

Output Format

Select Language : C (GCC 9.2.0)

```c
                    index = 1;
                }
            }
        }

        if(index == -1) {
            time++;
        }
        else {
            time += bt[index];
            ct[index] = time;

            tat[index] = ct[index] - at[index];
            wt[index] = tat[index] - bt[index];

            visited[index] = 1;
            completed++;
        }
    }
    float avgTAT = 0, avgWT = 0;

    for(i = 0; i < n; i++) {
        avgTAT += tat[i];
        avgWT += wt[i];
    }
```

Test Cases 2

Run Sample Cases

Run All Cases

Back To Practice

Q1

Save

Completed

Non-Preemptive Priority Scheduling algorithm

Write a C program to implement the Non-Preemptive Priority Scheduling algorithm. The program should accept number of processes their Arrival Time (AT), Burst Time (BT), and Priority for each process. The process with the highest priority (lowest numerical value) must be executed first among those that have arrived. Calculate the Average Waiting Time and Average Turnaround Time.

Input Format

The first line contains an integer representing the number of processes.

The next lines contain three integers for each process, where

- The first integer denotes the Arrival Time (AT)
- The second integer denotes the Burst Time (BT)
- The third integer denotes the Priority of the process.

A lower numerical value indicates a higher priority.

Output Format

Select Language : C (GCC 9.2.0)

```c
            ct[index] = time;

            tat[index] = ct[index] - at[index];
            wt[index] = tat[index] - bt[index];

            visited[index] = 1;
            completed++;
        }
    }
    float avgTAT = 0, avgWT = 0;

    for(i = 0; i < n; i++) {
        avgTAT += tat[i];
        avgWT += wt[i];
    }

    avgTAT /= n;
    avgWT /= n;

    printf("\nAverage Turnaround Time = %.2f\n", avgTAT);
    printf("Average Waiting Time = %.2f\n", avgWT);

    return 0;
}
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

Custom Test

Success

Input :

```
3
0 5 2
1 3 1
2 2 3
```

Expected Output :

```
Enter number of processes:
Enter AT, BT and Priority for P1:
Enter AT, BT and Priority for P2:
Enter AT, BT and Priority for P3:

Average Turnaround Time = 6.67
Average Waiting Time = 3.33
```

Output :

```
Enter number of processes:
Enter AT, BT and Priority for P1:
Enter AT, BT and Priority for P2:
Enter AT, BT and Priority for P3:

Average Turnaround Time = 6.67
Average Waiting Time = 3.33
```

Sample Test Case

Test Case - 1 Success

Input :

```
3
0 5 2
1 3 1
2 2 3
```

Expected Output :

```
Enter number of processes:
Enter AT, BT and Priority for P1:
Enter AT, BT and Priority for P2:
Enter AT, BT and Priority for P3:

Average Turnaround Time = 6.67
Average Waiting Time = 3.33
```

Output :

```
Enter number of processes:
Enter AT, BT and Priority for P1:
Enter AT, BT and Priority for P2:
Enter AT, BT and Priority for P3:

Average Turnaround Time = 6.67
Average Waiting Time = 3.33
```

Test Case - 2 Success

Input :

```
4
0 4 3
0 3 1
2 2 4
5 1 2
```

Expected Output :

```
Enter number of processes:
Enter AT, BT and Priority for P1:
Enter AT, BT and Priority for P2:
Enter AT, BT and Priority for P3:
Enter AT, BT and Priority for P4:

Average Turnaround Time = 5.25
Average Waiting Time = 2.75
```

Output :

```
Enter number of processes:
Enter AT, BT and Priority for P1:
Enter AT, BT and Priority for P2:
Enter AT, BT and Priority for P3:
Enter AT, BT and Priority for P4:

Average Turnaround Time = 5.25
Average Waiting Time = 2.75
```